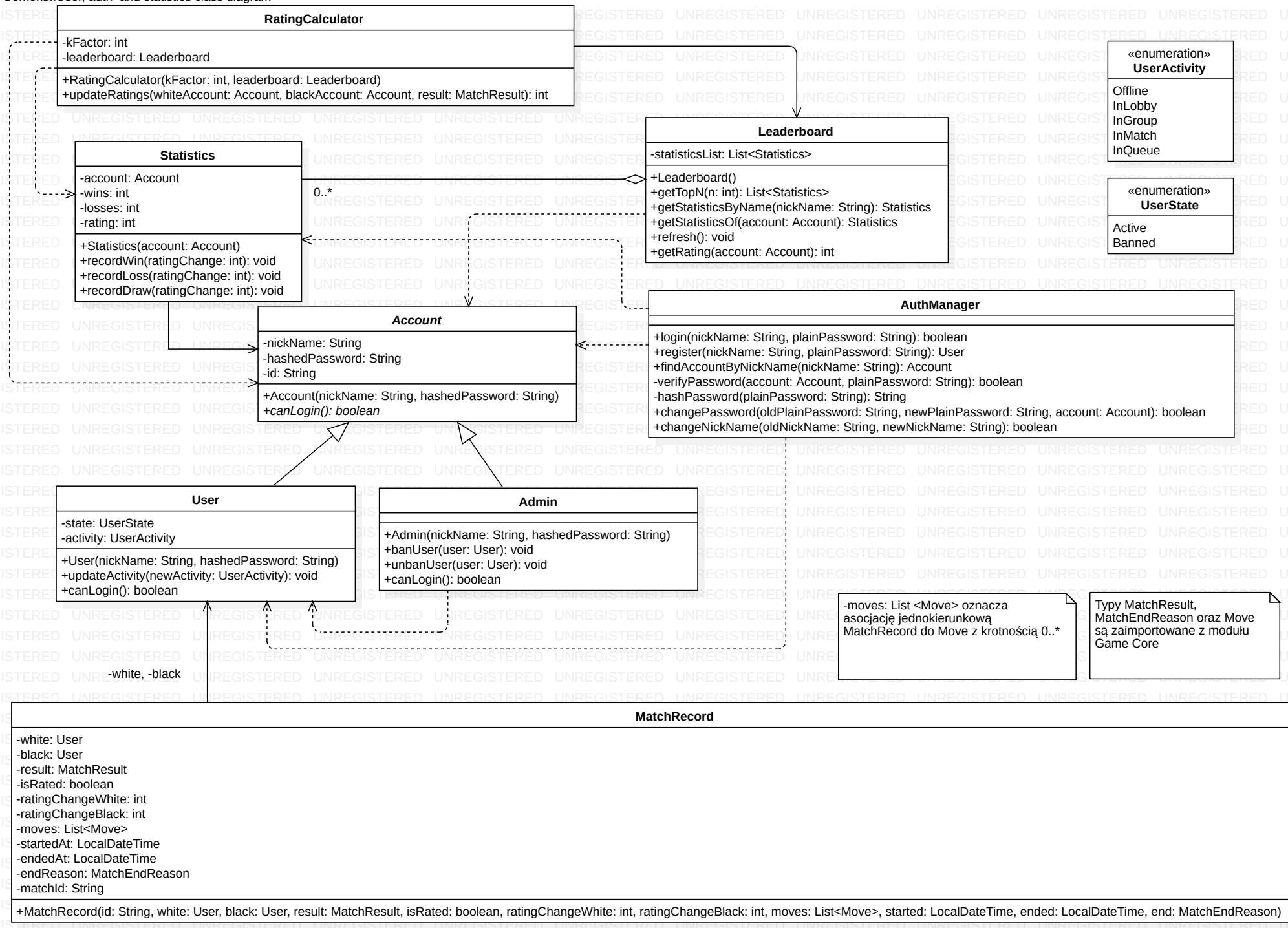
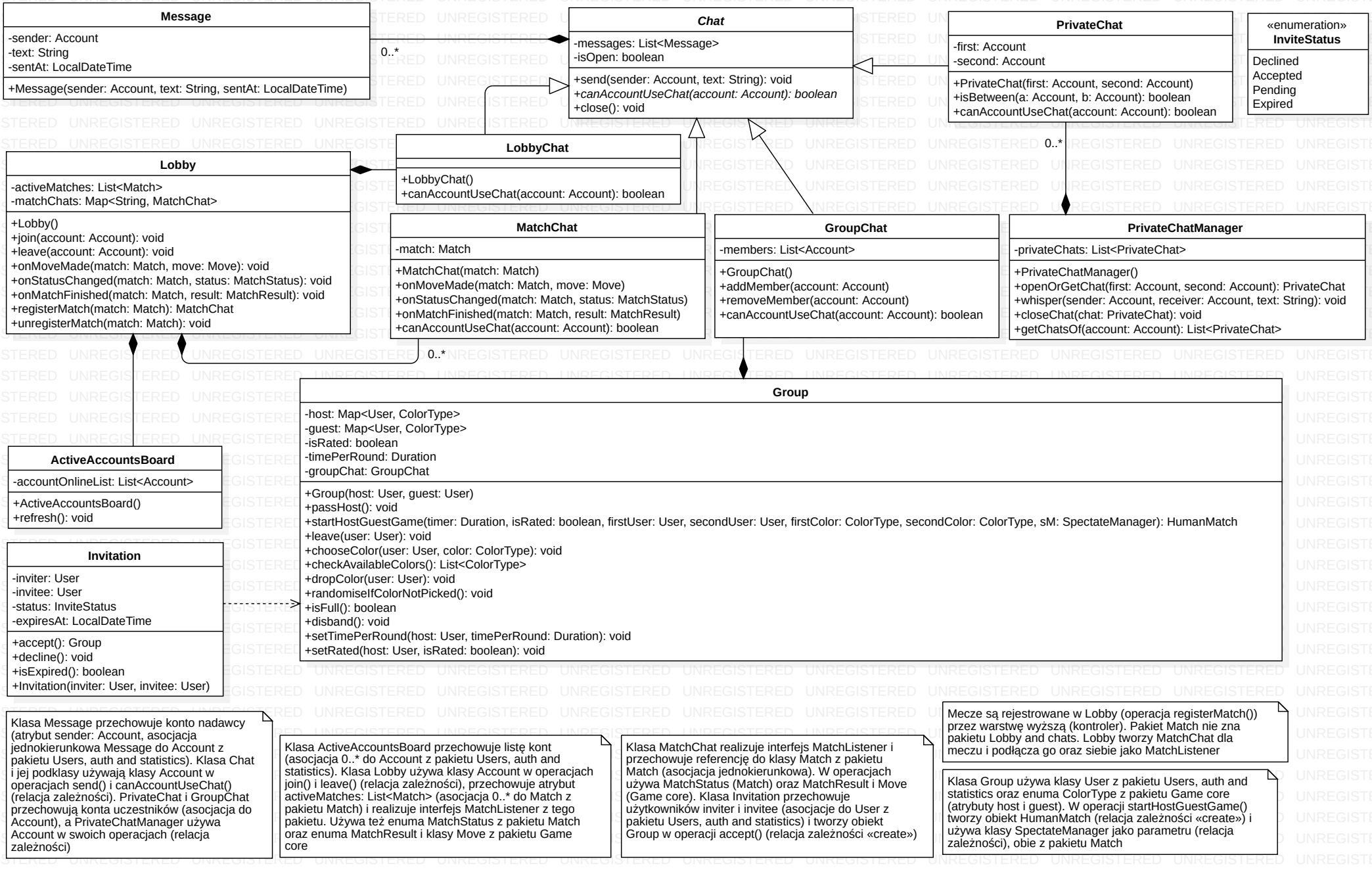
Gomoku::User, auth and statistics class diagram



Gomoku::Lobby and chats class diagram



Klasa Message przechowuje konto nadawcy (atrybut sender: Account, asocjacja jednokierunkowa Message do Account z pakietu Users, auth and statistics). Klasa Chat i jej podklasy używają klasy Account w operacjach send() i canAccountUseChat() (relacja zależności). PrivateChat i GroupChat przechowują konta uczestników (asocjacja do Account), a PrivateChatManager używa Account w swoich operacjach (relacja zależności)

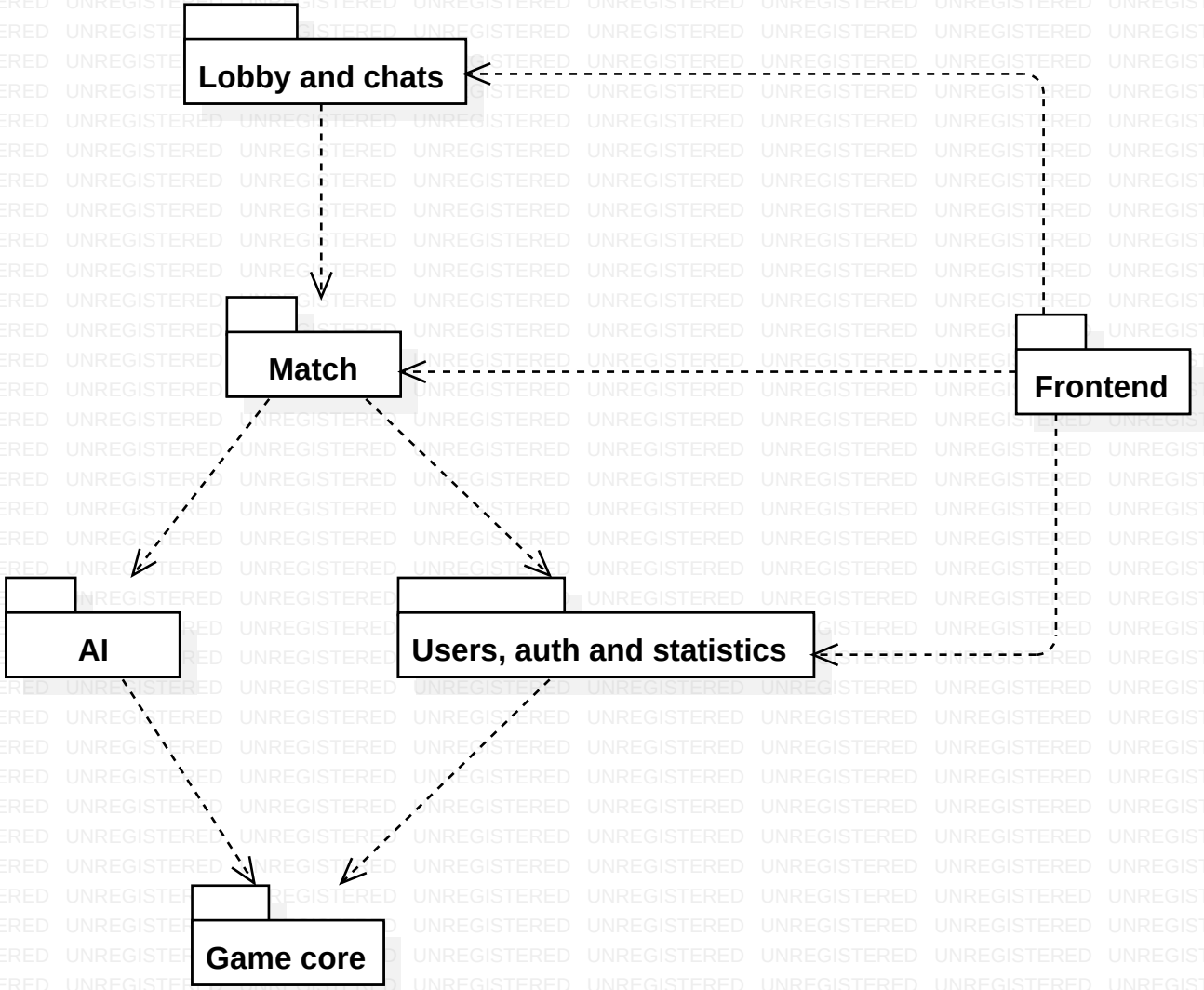
Klasa ActiveAccountsBoard przechowuje listę kont (asocjacja 0..* do Account z pakietu Users, auth and statistics). Klasa Lobby używa klasy Account w operacjach join() i leave() (relacja zależności), przechowuje atrybut activeMatches: List<Match> (asocjacja 0..* do Match z pakietu Match) i realizuje interfejs MatchListener z tego pakietu. Używa też enuma MatchStatus z pakietu Match oraz enuma MatchResult i klasy Move z pakietu Game core

Klasa MatchChat realizuje interfejs MatchListener i przechowuje referencję do klasy Match z pakietu Match (asocjacja jednokierunkowa). W operacjach używa MatchStatus (Match) oraz MatchResult i Move (Game core). Klasa Invitation przechowuje użytkowników inviter i invitee (asocjacje do User z pakietu Users, auth and statistics) i tworzy obiekt Group w operacji accept() (relacja zależności «create»)

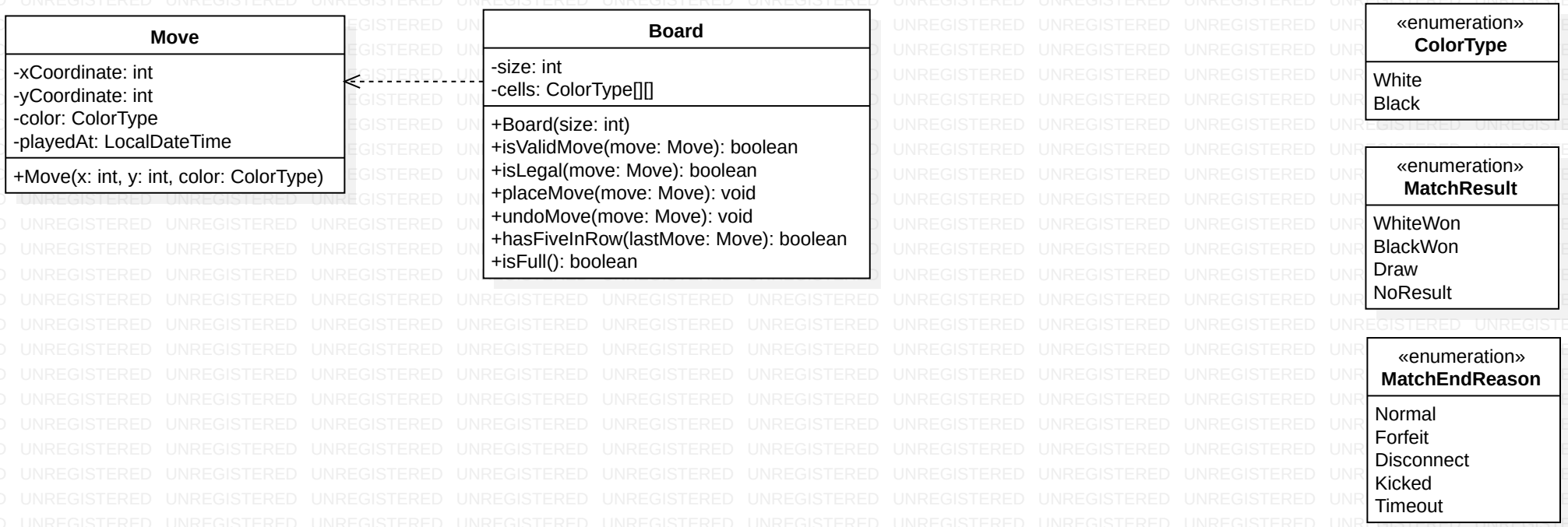
Mecze są rejestrowane w Lobby (operacja registerMatch()) przez warstwę wyższą (kontroler). Pakiet Match nie zna pakietu Lobby and chats. Lobby tworzy MatchChat dla meczu i podłącza go oraz siebie jako MatchListener

Klasa Group używa klasy User z pakietu Users, auth and statistics oraz enuma ColorType z pakietu Game core (atrybuty host i guest). W operacji startHostGuestGame() tworzy obiekt HumanMatch (relacja zależności «create») i używa klasy SpectateManager jako parametru (relacja zależności), obie z pakietu Match

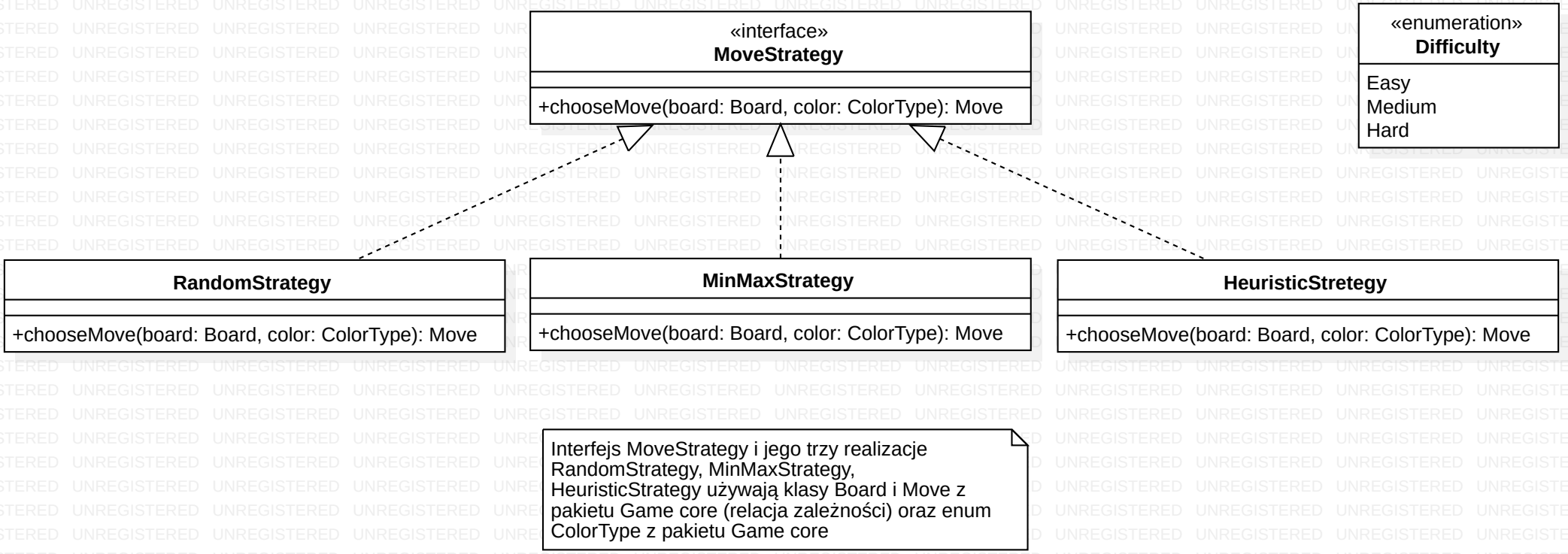
Gomoku::Package diagram



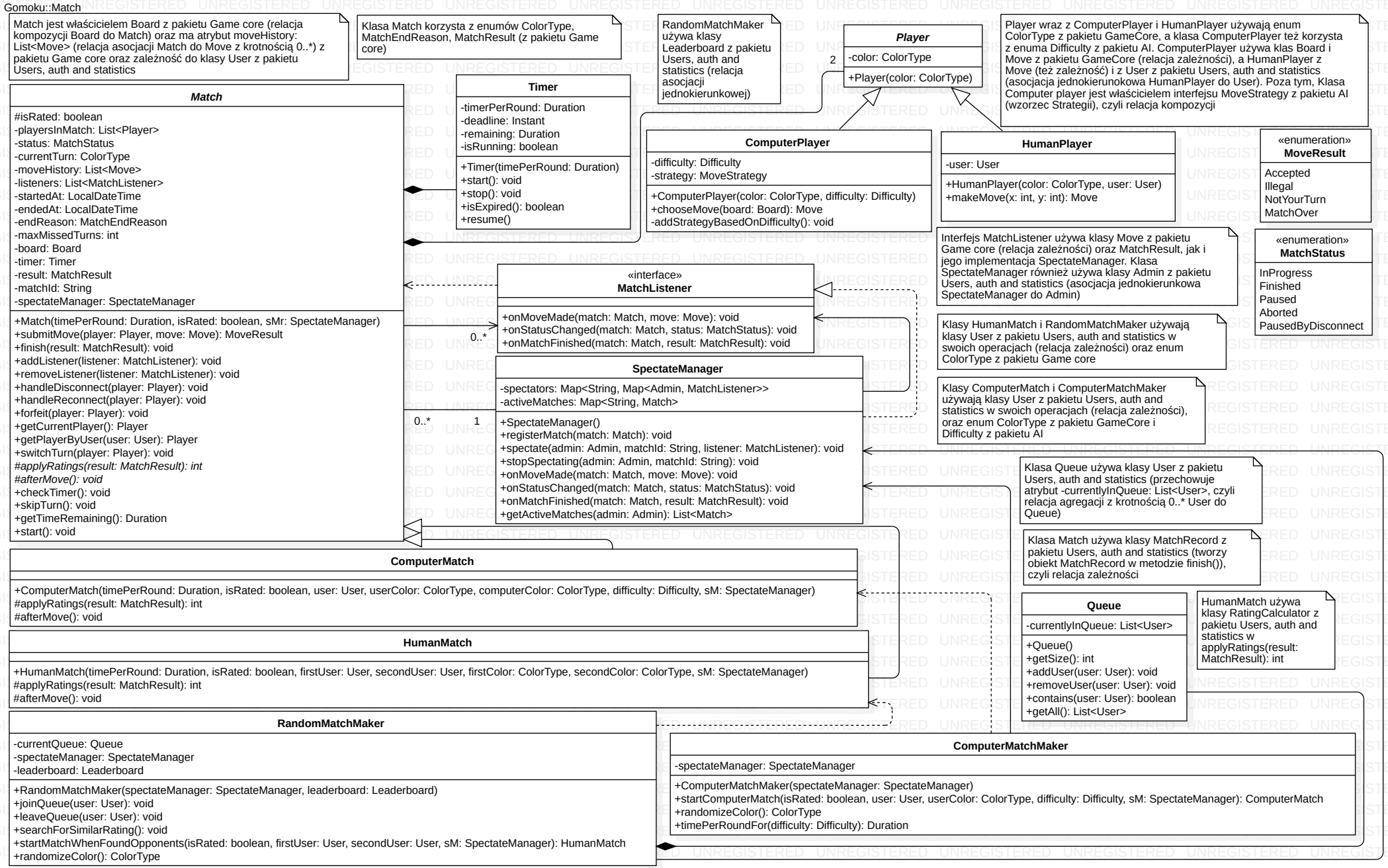
Gomoku::Game core class diagram



Gomoku::AI



Interfejs MoveStrategy i jego trzy realizacje RandomStrategy, MinMaxStrategy, HeuristicStrategy używają klasy Board i Move z pakietu Game core (relacja zależności) oraz enum ColorType z pakietu Game core


Gomoku::Match
Match jest właścicielem Board z pakietu Game core (relacja kompozycji Board do Match) oraz ma atrybut moveHistory: List<Move> (relacja asocjacji Match do Move z krotnością 0..*) z pakietu Game core oraz zależność do klasy User z pakietu Users, auth and statistics
Klasa Match korzysta z enumów ColorType, MatchEndReason, MatchResult (z pakietu Game core)
RandomMatchMaker używa klasy Leaderboard z pakietu Users, auth and statistics (relacja asocjacji jednokierunkowej)
Player wraz z ComputerPlayer i HumanPlayer używają enum ColorType z pakietu GameCore, a klasa ComputerPlayer też korzysta z enuma Difficulty z pakietu AI. ComputerPlayer używa klas Board i Move z pakietu GameCore (relacja zależności), a HumanPlayer z Move (też zależność) i z User z pakietu Users, auth and statistics (asocjacja jednokierunkowa HumanPlayer do User). Poza tym, Klasa Computer player jest właścicielem interfejsu MoveStrategy z pakietu AI (wzorzec Strategii), czyli relacja kompozycji
Match
#isRated: boolean
-playersInMatch: List<Player>
-status: MatchStatus
-currentTurn: ColorType
-moveHistory: List<Move>
-listeners: List<MatchListener>
-startedAt: LocalDateTime
-endedAt: LocalDateTime
-endReason: MatchEndReason
-maxMissedTurns: int
-board: Board
-timer: Timer
-result: MatchResult
-matchId: String
-spectateManager: SpectateManager
+Match(timePerRound: Duration, isRated: boolean, sMr: SpectateManager)
+submitMove(player: Player, move: Move): MoveResult
+finish(result: MatchResult): void
+addListener(listener: MatchListener): void
+removeListener(listener: MatchListener): void
+handleDisconnect(player: Player): void
+handleReconnect(player: Player): void
+forfeit(player: Player): void
+getCurrentPlayer(): Player
+getPlayerByUser(user: User): Player
+switchTurn(player: Player): void
#applyRatings(result: MatchResult): int
#afterMove(): void
+checkTimer(): void
+skipTurn(): void
+getTimeRemaining(): Duration
+start(): void
Timer
-timerPerRound: Duration
-deadline: Instant
-remaining: Duration
-isRunning: boolean
+Timer(timePerRound: Duration)
+start(): void
+stop(): void
+isExpired(): boolean
+resume()
Player
-color: ColorType
+Player(color: ColorType)
2
ComputerPlayer
-difficulty: Difficulty
-strategy: MoveStrategy
+ComputerPlayer(color: ColorType, difficulty: Difficulty)
+chooseMove(board: Board): Move
-addStrategyBasedOnDifficulty(): void
HumanPlayer
-user: User
+HumanPlayer(color: ColorType, user: User)
+makeMove(x: int, y: int): Move
«enumeration»
MoveResult
Accepted
Illegal
NotYourTurn
MatchOver
«enumeration»
MatchStatus
InProgress
Finished
Paused
Aborted
PausedByDisconnect
«interface»
MatchListener
+onMoveMade(match: Match, move: Move): void
+onStatusChanged(match: Match, status: MatchStatus): void
+onMatchFinished(match: Match, result: MatchResult): void
0..*
Interfejs MatchListener używa klasy Move z pakietu Game core (relacja zależności) oraz MatchResult, jak i jego implementacja SpectateManager. Klasa SpectateManager również używa klasy Admin z pakietu Users, auth and statistics (asocjacja jednokierunkowa SpectateManager do Admin)
Klasy HumanMatch i RandomMatchMaker używają klasy User z pakietu Users, auth and statistics w swoich operacjach (relacja zależności) oraz enum ColorType z pakietu Game core
Klasy ComputerMatch i ComputerMatchMaker używają klasy User z pakietu Users, auth and statistics w swoich operacjach (relacja zależności), oraz enum ColorType z pakietu GameCore i Difficulty z pakietu AI
SpectateManager
-spectators: Map<String, Map<Admin, MatchListener>>
-activeMatches: Map<String, Match>
+SpectateManager()
+registerMatch(match: Match): void
+spectate(admin: Admin, matchId: String, listener: MatchListener): void
+stopSpectating(admin: Admin, matchId: String): void
+onMoveMade(match: Match, move: Move): void
+onStatusChanged(match: Match, status: MatchStatus): void
+onMatchFinished(match: Match, result: MatchResult): void
+getActiveMatches(admin: Admin): List<Match>
0..*
1
Klasa Queue używa klasy User z pakietu Users, auth and statistics (przechowuje atrybut -currentlyInQueue: List<User>, czyli relacja agregacji z krotnością 0..* User do Queue)
Klasa Match używa klasy MatchRecord z pakietu Users, auth and statistics (tworzy obiekt MatchRecord w metodzie finish()), czyli relacja zależności
Queue
-currentlyInQueue: List<User>
+Queue()
+getSize(): int
+addUser(user: User): void
+removeUser(user: User): void
+contains(user: User): boolean
+getAll(): List<User>
HumanMatch używa klasy RatingCalculator z pakietu Users, auth and statistics w applyRatings(result: MatchResult): int
ComputerMatch
+ComputerMatch(timePerRound: Duration, isRated: boolean, user: User, userColor: ColorType, computerColor: ColorType, difficulty: Difficulty, sM: SpectateManager)
#applyRatings(result: MatchResult): int
#afterMove(): void
HumanMatch
+HumanMatch(timePerRound: Duration, isRated: boolean, firstUser: User, secondUser: User, firstColor: ColorType, secondColor: ColorType, sM: SpectateManager)
#applyRatings(result: MatchResult): int
#afterMove(): void
RandomMatchMaker
-currentQueue: Queue
-spectateManager: SpectateManager
-leaderboard: Leaderboard
+RandomMatchMaker(spectateManager: SpectateManager, leaderboard: Leaderboard)
+joinQueue(user: User): void
+leaveQueue(user: User): void
+searchForSimilarRating(): void
+startMatchWhenFoundOpponents(isRated: boolean, firstUser: User, secondUser: User, sM: SpectateManager): HumanMatch
+randomizeColor(): ColorType
ComputerMatchMaker
-spectateManager: SpectateManager
+ComputerMatchMaker(spectateManager: SpectateManager)
+startComputerMatch(isRated: boolean, user: User, userColor: ColorType, difficulty: Difficulty, sM: SpectateManager): ComputerMatch
+randomizeColor(): ColorType
+timePerRoundFor(difficulty: Difficulty): Duration